

LAPORAN PROGRESS REPORT
SECURE DIGITAL WALLET
Simulasi Sistem Keamanan Dompet Digital
Berbasis Python, Tkinter, dan SQLite

Oleh :

Arya Rayan Utama (F1D02310106)

Lutfi Alfarizi (F1D02310121)

PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi yang pesat telah mendorong transformasi besar pada sistem keuangan digital. Penggunaan dompet digital (digital wallet) semakin luas sebagai alternatif transaksi non-tunai yang praktis, cepat, dan efisien. Namun, seiring pertumbuhannya, ancaman keamanan seperti pencurian data, penyadapan transaksi, pemalsuan identitas, dan serangan terhadap basis data juga semakin meningkat.

Untuk mengatasi permasalahan tersebut, dikembangkan program Secure Digital Wallet sebagai simulasi sistem dompet digital yang menerapkan beberapa lapisan keamanan kriptografi. Program ini dibangun menggunakan Python dengan antarmuka Tkinter, basis data SQLite, serta mengimplementasikan algoritma AES-256, RSA-2048, dan SHA-256 untuk melindungi data pengguna dan transaksi..

Selain itu, sistem menerapkan role-based access control yang membedakan hak akses antara pengguna dan administrator. Pengguna dapat melakukan transfer saldo, sedangkan administrator memiliki kewenangan melakukan top up saldo. Setiap transaksi juga memerlukan konfirmasi PIN sebagai lapisan autentikasi tambahan untuk meningkatkan keamanan sistem.

B. Deskripsi Aplikasi dan Tujuan Proyek

1. Deskripsi Aplikasi

Secure Digital Wallet merupakan aplikasi desktop yang disimulasikan sebagai dompet digital dengan keamanan berlapis. Aplikasi dibangun menggunakan Python dengan GUI Tkinter, database SQLite, serta library pycryptodome untuk implementasi algoritma kriptografi. Seluruh data sensitif yang tersimpan di database, termasuk saldo pengguna dan detail setiap transaksi, dienkripsi menggunakan AES-256 sebelum disimpan sehingga tidak dapat dibaca langsung meskipun file database berhasil diakses secara langsung.

Aplikasi memiliki dua halaman dashboard yang berbeda berdasarkan peran pengguna. Dashboard pengguna biasa menampilkan informasi saldo, fitur transfer, riwayat transaksi, dan panel informasi kriptografi. Dashboard administrator menampilkan ringkasan sistem, formulir top up saldo, daftar semua transaksi, dan daftar pengguna terdaftar.

2. Tujuan Proyek

Pengembangan proyek ini memiliki beberapa tujuan utama, antara lain:

- Mengimplementasikan algoritma kriptografi SHA-256 untuk hashing password dan PIN pengguna sehingga tidak ada data autentikasi yang tersimpan dalam bentuk plaintext.
- Menerapkan enkripsi AES-256 dengan mode CBC pada seluruh data sensitif yang tersimpan di database SQLite, meliputi saldo pengguna dan detail transaksi.
- Mengintegrasikan simulasi RSA-2048 sebagai mekanisme perlindungan kunci enkripsi AES (key wrapping) untuk mendemonstrasikan konsep kriptografi bertingkat.
- Membangun sistem autentikasi berlapis yang terdiri dari verifikasi password saat login dan verifikasi PIN saat eksekusi transaksi.
- Mengembangkan sistem manajemen pengguna dengan dua level hak akses (role-based access control), yaitu peran Admin dan peran User.
- Menyediakan antarmuka grafis berbasis Tkinter yang modern, intuitif, dan mudah digunakan untuk seluruh fitur yang tersedia.
- Menjamin integritas dan kerahasiaan data melalui mekanisme enkripsi-dekripsi transparan di latar belakang tanpa mengganggu pengalaman pengguna.
- Mengimplementasikan materi kriptografi ke dalam proyek nyata berbasis GUI desktop.

STRUKTUR PROJECT DAN PERANCANGAN SISTEM

A. Struktur Project dan Penerapan Separation of Concerns (SoC)

Aplikasi Secure Digital Wallet dibangun dengan mengikuti prinsip Separation of Concerns (SoC), di mana program dibagi menjadi beberapa modul berbeda berdasarkan tanggung jawab spesifiknya. Hal ini mencegah terjadinya kode yang saling tumpang tindih dan sulit dipelihara. Struktur direktori aplikasi disusun sebagai berikut:

Tabel 1. Struktur Direktori Aplikasi

Modul / File	Deskripsi Tanggung Jawab
main.py	Entry point aplikasi. Mengelola seluruh tampilan GUI Tkinter, routing halaman login/register/dashboard, dan seluruh event pengguna.
auth.py	Logika autentikasi: hashing password dan PIN dengan SHA-256, validasi login, registrasi pengguna baru, dan verifikasi PIN.
database.py	Abstraksi akses database SQLite. Mengelola operasi CRUD untuk tabel users dan transactions, enkripsi/dekripsi data secara otomatis.
encryption.py	Implementasi kriptografi: AES-256 CBC untuk enkripsi data, RSA-2048 OAEP untuk proteksi kunci AES, dan manajemen kunci RSA.
transaction.py	Logika bisnis transaksi: validasi dan eksekusi transfer saldo antar pengguna serta proses top up saldo oleh administrator.
assets/wallet.db	File database SQLite yang menyimpan data pengguna dan riwayat transaksi dalam bentuk terenkripsi.
assets/rsa_keys.json	File penyimpanan pasangan kunci RSA-2048 (public dan private key) dalam format PEM.
assets/aes_key.enc	File penyimpanan kunci AES-256 yang telah dienkripsi menggunakan RSA public key.

B. Desain Database

Aplikasi menggunakan SQLite sebagai sistem manajemen basis data karena sifatnya yang serverless dan terintegrasi langsung dalam file lokal. Terdapat dua tabel utama yang dirancang untuk menyimpan data pengguna dan riwayat transaksi. Seluruh data sensitif dienkripsi menggunakan AES-256 sebelum disimpan.

Tabel 2. Struktur Tabel *users*

Kolom	Tipe Data	Keterangan
id	INTEGER	Primary key, auto increment
username	TEXT	Nama pengguna unik (plaintext)
password_hash	TEXT	SHA-256 hash dari (salt + password)
pin_hash	TEXT	SHA-256 hash dari (pin_salt + PIN), nullable
balance_enc	TEXT	Saldo pengguna dienkripsi AES-256 CBC
role	TEXT	Peran pengguna: 'admin' atau 'user'
created_at	TEXT	Timestamp registrasi akun

Tabel 3. Struktur Tabel *transactions*

Kolom	Tipe Data	Keterangan
id	INTEGER	Primary key, auto increment
sender	TEXT	Username pengirim (plaintext)
receiver	TEXT	Username penerima (plaintext)
amount_enc	TEXT	Jumlah transaksi dienkripsi AES-256 CBC
memo_enc	TEXT	Keterangan transaksi dienkripsi AES-256 CBC
tx_type	TEXT	Jenis transaksi: 'transfer' atau 'topup'
timestamp	TEXT	Waktu transaksi (datetime lokal)

C. Alur Sistem

1. Alur Registrasi Pengguna Baru

Ketika pengguna mendaftar akun baru, sistem akan memvalidasi seluruh input yang diberikan. Username hanya boleh terdiri dari huruf dan angka dengan minimal 3 karakter, sedangkan password minimal 6 karakter. Setelah validasi berhasil, password akan di-hash menggunakan SHA-256 dengan salt tetap sebelum disimpan ke database. Akun baru dibuat dengan saldo awal Rp 0. Setelah akun berhasil dibuat, pengguna diarahkan ke halaman Setup PIN untuk membuat PIN transaksi yang bersifat wajib dan terdiri dari minimal 4 digit angka. PIN tersebut juga di-hash menggunakan SHA-256 dengan salt berbeda sebelum disimpan.

2. Alur Login dan Autentikasi

Saat pengguna melakukan login, sistem mengambil data hash password yang tersimpan di database kemudian membandingkannya dengan hash dari password yang dimasukkan. Jika hash cocok, sistem menentukan peran pengguna (admin atau user) dan mengarahkan ke dashboard yang sesuai. Seluruh data dashboard di-refresh secara otomatis saat login sehingga tidak ada data sisa dari sesi sebelumnya.

3. Alur Transfer Saldo

Sebelum transfer dieksekusi, sistem melakukan serangkaian validasi: memastikan penerima terdaftar dan bukan akun admin, memastikan jumlah transfer lebih dari nol, dan memastikan saldo pengirim mencukupi. Setelah validasi berhasil, dialog PIN ditampilkan untuk konfirmasi identitas pengguna. Jika PIN benar, saldo pengirim dikurangi dan saldo penerima ditambah secara atomik. Detail transaksi (jumlah dan memo) dienkripsi AES-256 sebelum disimpan ke database.

4. Alur Top Up oleh Administrator

Administrator dapat melakukan top up saldo ke akun pengguna mana pun melalui form khusus di dashboard admin. Administrator memilih pengguna target dari dropdown yang menampilkan saldo terkini, mengisi jumlah top up dan keterangan, kemudian mengkonfirmasi. Sistem mencatat transaksi dengan tipe 'topup' sehingga dapat dibedakan dari transaksi transfer biasa pada riwayat transaksi.

KODE PROGRAM DAN PENJELASAN

A. Modul Enkripsi (encryption.py)

1. Enkripsi AES-256 CBC

Fungsi `aes_encrypt()` mengimplementasikan enkripsi simetris AES-256 dengan mode CBC. Setiap kali fungsi dipanggil, sebuah IV (Initialization Vector) sepanjang 16 byte dibangkitkan secara acak menggunakan `get_random_bytes()`. IV yang unik pada setiap enkripsi memastikan bahwa nilai yang sama akan menghasilkan ciphertext yang berbeda, sehingga tahan terhadap pola yang dapat dieksploitasi. Hasil enkripsi berupa byte IV yang digabungkan dengan ciphertext, kemudian di-encode ke Base64 untuk penyimpanan di database.

Kode 1. Fungsi Enkripsi dan Dekripsi AES-256 CBC

```
def aes_encrypt(plaintext: str, aes_key: bytes) -> str:
    """
    Mengenkripsi string plaintext dengan AES-256 CBC.
    IV acak di-prefix ke ciphertext sebelum di-encode Base64.
    Returns: base64-encoded (IV + ciphertext)
    """
    iv = get_random_bytes(16)          # IV 16 byte acak
    cipher = AES.new(aes_key, AES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(
        pad(plaintext.encode(), AES.block_size)
    )
    return base64.b64encode(iv + ciphertext).decode()

def aes_decrypt(encrypted_b64: str, aes_key: bytes) -> str:
    """
    Mendekripsi data yang dienkripsi dengan aes_encrypt().
    IV diambil dari 16 byte pertama setelah Base64 decode.
    """
    raw = base64.b64decode(encrypted_b64)
    iv = raw[:16]                      # Ekstrak IV
    ciphertext = raw[16:]
    cipher = AES.new(aes_key, AES.MODE_CBC, iv)
    plaintext = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return plaintext.decode()
```

2. Proteksi Kunci AES dengan RSA-2048

Kunci AES-256 tidak pernah disimpan dalam bentuk plaintext. Setiap kali program pertama kali dijalankan, pasangan kunci RSA-2048 dibangkitkan dan kunci AES dienkripsi

menggunakan RSA public key dengan skema padding OAEP (Optimal Asymmetric Encryption Padding). Kunci AES yang terenkripsi inilah yang disimpan ke file. Saat dibutuhkan, kunci AES tersebut didekripsi kembali menggunakan RSA private key.

Kode 2. Fungsi Pengelolaan Kunci AES dengan RSA

```
def get_or_create_wallet_aes_key() -> bytes:
    """
    Mengambil atau membuat AES key global untuk wallet.
    AES key dienkripsi RSA public key sebelum disimpan.
    """
    public_key, private_key = load_rsa_keys()
    os.makedirs('assets', exist_ok=True)

    if not os.path.exists(AES_KEY_FILE):
        # Buat AES key baru (32 byte = 256 bit)
        aes_key = generate_aes_key()
        # Enkripsi AES key dengan RSA public key (OAEP)
        encrypted = rsa_encrypt_aes_key(aes_key, public_key)
        with open(AES_KEY_FILE, 'w') as f:
            f.write(encrypted)
        return aes_key

    # Baca dan dekripsi AES key yang tersimpan
    with open(AES_KEY_FILE, 'r') as f:
        encrypted = f.read()
    return rsa_decrypt_aes_key(encrypted, private_key)
```

B. Modul Autentikasi (auth.py)

1. Hashing Password dan PIN dengan SHA-256

Password dan PIN pengguna tidak pernah disimpan dalam bentuk aslinya. Keduanya di-hash menggunakan SHA-256 dengan salt yang berbeda. Penggunaan dua salt yang berbeda memastikan bahwa meskipun pengguna menggunakan PIN yang sama dengan password-nya, nilai hash yang tersimpan di database akan berbeda sehingga tidak dapat saling dikaitkan.

Kode 3. Fungsi Hashing SHA-256 untuk Password dan PIN

```
def hash_password(password: str) -> str:
    """
    SHA-256 hash dari password dengan salt khusus password.
    """
    salt = 'secure_wallet_salt_v1'
    salted = salt + password
```



```

        return hashlib.sha256(salted.encode()).hexdigest()

def hash_pin(pin: str) -> str:
    """
    SHA-256 hash dari PIN dengan salt berbeda dari password.
    Salt yang berbeda mencegah korelasi antara hash PIN dan hash password.
    """
    salt = 'secure_wallet_pin_salt_v1'
    salted = salt + pin
    return hashlib.sha256(salted.encode()).hexdigest()

```

2. Proses Registrasi dan Setup PIN

Fungsi `register()` melakukan serangkaian validasi sebelum akun dibuat: username minimal 3 karakter dan hanya boleh alphanumeric, password minimal 6 karakter, dan username tidak boleh 'admin' karena sudah digunakan sistem. Fungsi `setup_pin()` dipanggil setelah registrasi berhasil untuk mewajibkan pengguna membuat PIN transaksi.

Kode 4. Fungsi Registrasi dan Setup PIN

```

def register(username: str, password: str) -> tuple:
    if len(username) < 3:
        return False, 'Username minimal 3 karakter.'
    if len(password) < 6:
        return False, 'Password minimal 6 karakter.'
    if not username.isalnum():
        return False, 'Username hanya boleh berisi huruf dan angka.'
    if username.lower() == 'admin':
        return False, "Username 'admin' sudah digunakan sistem."
    password_hash = hash_password(password)
    success = create_user(username, password_hash)
    if success:
        return True, f"Akun '{username}' berhasil dibuat!"
    return False, f"Username '{username}' sudah digunakan."

def setup_pin(username: str, pin: str, pin_confirm: str) -> tuple:
    if len(pin) < 4:
        return False, 'PIN minimal 4 digit.'
    if not pin.isdigit():
        return False, 'PIN hanya boleh berisi angka.'
    if pin != pin_confirm:
        return False, 'Konfirmasi PIN tidak cocok.'
    set_pin(username, hash_pin(pin))
    return True, 'PIN berhasil disimpan.'

```

C. Modul Database (database.py)

1. Inisialisasi Database dan Akun Admin Bawaan

Fungsi `init_db()` membuat skema tabel `users` dan `transactions` jika belum ada. Setelah tabel dibuat, fungsi `_ensure_admin()` dipanggil secara otomatis untuk memastikan akun administrator dengan username `'admin'` dan password `'admin123'` selalu tersedia. Perintah `INSERT OR IGNORE` digunakan sehingga akun admin tidak dibuat ulang jika sudah ada.

Kode 5. Inisialisasi Database dan Pembuatan Akun Admin

```
def init_db():
    conn = get_connection()
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            username      TEXT      UNIQUE NOT NULL,
            password_hash TEXT      NOT NULL,
            pin_hash      TEXT,
            balance_enc   TEXT      NOT NULL,
            role          TEXT      NOT NULL DEFAULT 'user',
            created_at    TEXT      DEFAULT (datetime('now','localtime'))
        )
    ''')
    conn.commit()
    conn.close()
    _ensure_admin()      # Pastikan akun admin selalu ada

def _ensure_admin():
    import hashlib
    salt = 'secure_wallet_salt_v1'
    pwd_hash = hashlib.sha256((salt+'admin123').encode()).hexdigest()
    balance_enc = encrypt_value('0')
    conn = get_connection()
    c = conn.cursor()
    c.execute('''
        INSERT OR IGNORE INTO users
        (username, password_hash, balance_enc, role)
        VALUES (?, ?, ?, 'admin')
    ''', ('admin', pwd_hash, balance_enc))
    conn.commit()
    conn.close()
```

2. Enkripsi Otomatis Saat Menyimpan dan Membaca Data

Setiap fungsi yang menyimpan data sensitif ke database akan memanggil `encrypt_value()` secara otomatis sebelum eksekusi query. Demikian pula, setiap fungsi yang membaca data sensitif dari database akan memanggil `decrypt_value()` secara otomatis setelah

query dieksekusi. Hal ini membuat lapisan enkripsi bersifat transparan bagi modul lain yang menggunakan database.py.

Kode 6. Fungsi Penyimpanan dan Pembacaan Saldo dengan Enkripsi Otomatis

```
def update_balance(username: str, new_balance: float):
    """Memperbarui saldo – enkripsi AES otomatis sebelum simpan."""
    conn = get_connection()
    c = conn.cursor()
    balance_enc = encrypt_value(str(new_balance)) # Enkripsi
    c.execute(
        'UPDATE users SET balance_enc = ? WHERE username = ?',
        (balance_enc, username)
    )
    conn.commit()
    conn.close()

def get_balance(username: str) -> float:
    """Mengambil saldo – dekripsi AES otomatis setelah baca."""
    user = get_user(username)
    if user is None:
        return 0.0
    return float(decrypt_value(user['balance_enc'])) # Dekripsi
```

D. Modul Transaksi (transaction.py)

1. Logika Transfer Saldo Antar Pengguna

Fungsi transfer() melakukan serangkaian validasi sebelum eksekusi: pengirim tidak boleh sama dengan penerima, penerima harus terdaftar dan bukan akun admin, jumlah harus positif dan tidak melebihi batas maksimum, serta saldo pengirim harus mencukupi. Setelah semua validasi lulus, saldo diperbarui untuk kedua pihak secara berurutan dan transaksi dicatat ke database. PIN sudah diverifikasi di lapisan GUI sebelum memanggil fungsi ini.

Kode 7. Fungsi Transfer Saldo Antar Pengguna

```
def transfer(sender: str, receiver: str,
            amount: float, memo: str = '') -> tuple:
    if sender == receiver:
        return False, 'Tidak dapat mengirim ke diri sendiri.'
    if get_user(receiver) is None:
        return False, f"Pengguna '{receiver}' tidak ditemukan."
    if get_user_role(receiver) == 'admin':
        return False, 'Tidak dapat transfer ke akun admin.'
    if amount <= 0:
        return False, 'Jumlah transfer harus lebih dari 0.'
    sender_balance = get_balance(sender)
    if sender_balance < amount:
```

```

        return False, (
            f'Saldo tidak cukup. Saldo Anda: Rp {sender_balance:,.0f}'
        )
    receiver_balance = get_balance(receiver)
    update_balance(sender, sender_balance - amount)
    update_balance(receiver, receiver_balance + amount)
    record_transaction(sender, receiver, amount,
                       memo or 'Transfer', tx_type='transfer')
    return True, f"Transfer Rp {amount:,.0f} ke '@{receiver}' berhasil!"

```

E. Dialog PIN di Antarmuka (main.py)

Dialog PIN diimplementasikan sebagai class `PinDialog` yang merupakan subclass dari `tk.Toplevel`. Dialog bersifat modal (`grab_set()`) sehingga pengguna tidak dapat berinteraksi dengan jendela lain selama dialog terbuka. Hasil input PIN dikembalikan melalui atribut `self.result` setelah dialog ditutup.

Kode 8. Implementasi Dialog PIN Modal

```

class PinDialog(tk.Toplevel):
    """
    Modal dialog untuk memasukkan PIN transaksi.
    Hasil PIN dikembalikan via self.result (None = batal).
    """
    def __init__(self, parent, title='Konfirmasi PIN'):
        super().__init__(parent)
        self.result = None
        self.grab_set()      # Modal: blokir interaksi jendela lain
        ...
        self._pin_var = tk.StringVar()
        pin_entry = tk.Entry(self, show='*',
                             textvariable=self._pin_var)
        pin_entry.bind('<Return>', lambda e: self._confirm())

    def _confirm(self):
        self.result = self._pin_var.get()  # Simpan hasil PIN
        self.destroy()

    def _cancel(self):
        self.result = None  # Batal = None
        self.destroy()

# Cara penggunaan di fungsi transfer:
dlg = PinDialog(self, 'Konfirmasi PIN Transfer')
self.wait_window(dlg)      # Tunggu dialog ditutup
if dlg.result is None:
    return                  # Pengguna membatalkan
if not verify_pin(self.current_user, dlg.result):
    messagebox.showerror('PIN Salah', 'PIN yang Anda masukkan salah.')
    return
# Lanjutkan eksekusi transfer...

```

TINJAUAN PUSTAKA

A. Kriptografi Simetris — AES (Advanced Encryption Standard)

Advanced Encryption Standard (AES) merupakan algoritma enkripsi simetris yang ditetapkan oleh National Institute of Standards and Technology (NIST) pada tahun 2001 berdasarkan algoritma Rijndael yang dikembangkan oleh Joan Daemen dan Vincent Rijmen. AES beroperasi pada blok data berukuran 128-bit dengan panjang kunci 128, 192, atau 256 bit. Dalam program ini digunakan AES-256 yang berarti kunci enkripsi sepanjang 256 bit, memberikan kekuatan enkripsi yang sangat tinggi.

Mode operasi yang digunakan adalah CBC (Cipher Block Chaining). Pada mode CBC, setiap blok plaintext di-XOR dengan blok ciphertext sebelumnya sebelum dienkripsi. Blok pertama menggunakan Initialization Vector (IV) sebagai pengganti ciphertext sebelumnya. Penggunaan IV yang acak dan unik untuk setiap operasi enkripsi memastikan bahwa plaintext yang sama akan menghasilkan ciphertext yang berbeda, sehingga tahan terhadap serangan chosen-plaintext attack.

B. Kriptografi Asimetris — RSA (Rivest-Shamir-Adleman)

RSA adalah algoritma kriptografi kunci publik yang pertama kali dipublikasikan pada tahun 1977 oleh Ron Rivest, Adi Shamir, dan Leonard Adleman. Keamanan RSA didasarkan pada kesulitan komputasional memfaktorkan bilangan bulat besar (integer factorization problem). Dengan panjang kunci 2048-bit yang digunakan dalam program ini, RSA dianggap aman untuk keperluan pengamanan data dalam jangka waktu yang sangat lama.

Dalam sistem ini, RSA digunakan untuk mengimplementasikan pola hybrid encryption, yaitu mengamankan kunci AES menggunakan RSA. Teknik ini dikenal sebagai key wrapping atau key encapsulation dan merupakan praktik standar dalam sistem kriptografi modern seperti TLS, PGP, dan S/MIME. Padding yang digunakan adalah OAEP (Optimal Asymmetric Encryption Padding) yang lebih aman dibandingkan PKCS1 v1.5

C. Fungsi Hash — SHA-256 (Secure Hash Algorithm 256)

SHA-256 adalah fungsi hash kriptografi yang menghasilkan output (digest) berukuran 256-bit dari input berukuran sembarang. SHA-256 merupakan bagian dari keluarga SHA-2

yang dikembangkan oleh National Security Agency (NSA). Fungsi hash memiliki sifat satu arah (one-way function).

Dalam program ini SHA-256 diimplementasikan menggunakan modul hashlib bawaan Python dengan penambahan salt untuk mengurangi risiko serangan rainbow table. Meskipun untuk sistem produksi skala besar dianjurkan menggunakan fungsi hash yang dirancang khusus untuk password seperti bcrypt atau Argon2 yang memiliki work factor yang dapat dikonfigurasi, penggunaan SHA-256 dengan salt pada program ini sudah cukup memadai untuk tujuan simulasi dan pembelajaran.

D. SQLite sebagai Database Lokal

SQLite adalah sistem manajemen basis data relasional yang bersifat serverless, artinya seluruh database tersimpan dalam satu file tunggal pada sistem file lokal. SQLite dipilih dalam program ini karena kemudahan integrasinya dengan Python melalui modul sqlite3 bawaan, serta kesesuaiannya untuk aplikasi desktop. Meskipun SQLite sendiri tidak menyediakan enkripsi data at-rest, keamanan data dijamin oleh mekanisme enkripsi AES yang diimplementasikan di lapisan aplikasi sebelum data disimpan.

E. Tkinter sebagai Framework GUI

Tkinter adalah library antarmuka grafis standar Python yang merupakan pembungkus dari Tcl/Tk GUI toolkit. Tkinter dipilih karena merupakan library bawaan Python sehingga tidak memerlukan instalasi terpisah dan kompatibel dengan berbagai platform (Windows, Linux, macOS). Dalam program ini, Tkinter digunakan bersama widget ttk (themed Tkinter) untuk komponen tabel data (Treeview) dan scrollbar.

F. pycryptodome sebagai Library Kriptografi

Pycryptodome adalah library Python yang menyediakan implementasi berbagai algoritma kriptografi termasuk AES, RSA, SHA, dan banyak lainnya. Library ini merupakan fork dari PyCrypto yang aktif dikembangkan dan diperbarui. Dalam program ini, pycryptodome digunakan untuk implementasi AES-256 CBC, RSA-2048 OAEP, dan PKCS7 padding. Perintah instalasi: `pip install pycryptodome`.

DAFTAR PUSTAKA

- [1] Daemen, J. & Rijmen, V. (2002). The Design of Rijndael: AES – The Advanced Encryption Standard. Berlin: Springer-Verlag.
- [2] NIST. (2015). Federal Information Processing Standard 180-4: Secure Hash Standard. National Institute of Standards and Technology.
- [3] Pycryptodome. (2024). PyCryptodome 3.x Documentation. Diakses dari: <https://pycryptodome.readthedocs.io/>
- [4] Python Software Foundation. (2024). sqlite3 – DB-API 2.0 interface for SQLite databases. Diakses dari: <https://docs.python.org/3/library/sqlite3.html>
- [5] Katz, J. & Lindell, Y. (2020). Introduction to Modern Cryptography (3rd ed.). Boca Raton: CRC Press.
- [6] Stallings, W. (2022). Cryptography and Network Security: Principles and Practice (8th ed.). Hoboken: Pearson Education.